

Term Project 2025 Report

Tero Kolari

Introduction

The objective of this project was to train a classifier to predict whether a *new particle formation* event (NPF) happens of a given day and if one does, what kind of event. The classifier will be trained with atmospheric measurement data from a forestry field station. As an additional challenge, the classifier is supposed to be trained with no information about the time or date of the measurements. Even though I am allowed to change my approach after the better teams showed their methods, the instructions do say I shouldn't "simply copy the method used by the teams with [better] performance". I therefore chose as my personal goal to train one model to make both binary and multi-class predictions, instead of using one model for predicting whether an event happens and if one does, another model for predicting which kind.

Data Analysis

The data consists of 450 observations each with 100 different features. There is one multi-class target with four values, **nonevent**, **II**, **Ia** and **Ib**. Exactly half of the observations belong in class **nonevent** and in the remaining half that are events, the split is 117, 82, 26, respectively. The data is relatively clean as there are no missing values. One of the features, 'partlybad', has the same value for each observation, so it can be ignored from further analysis and training. To visualize the data, and especially the distributions for each class, I plotted a histogram of every single feature with the data split into **event** and **nonevent** observations, which can be found in figure 1.

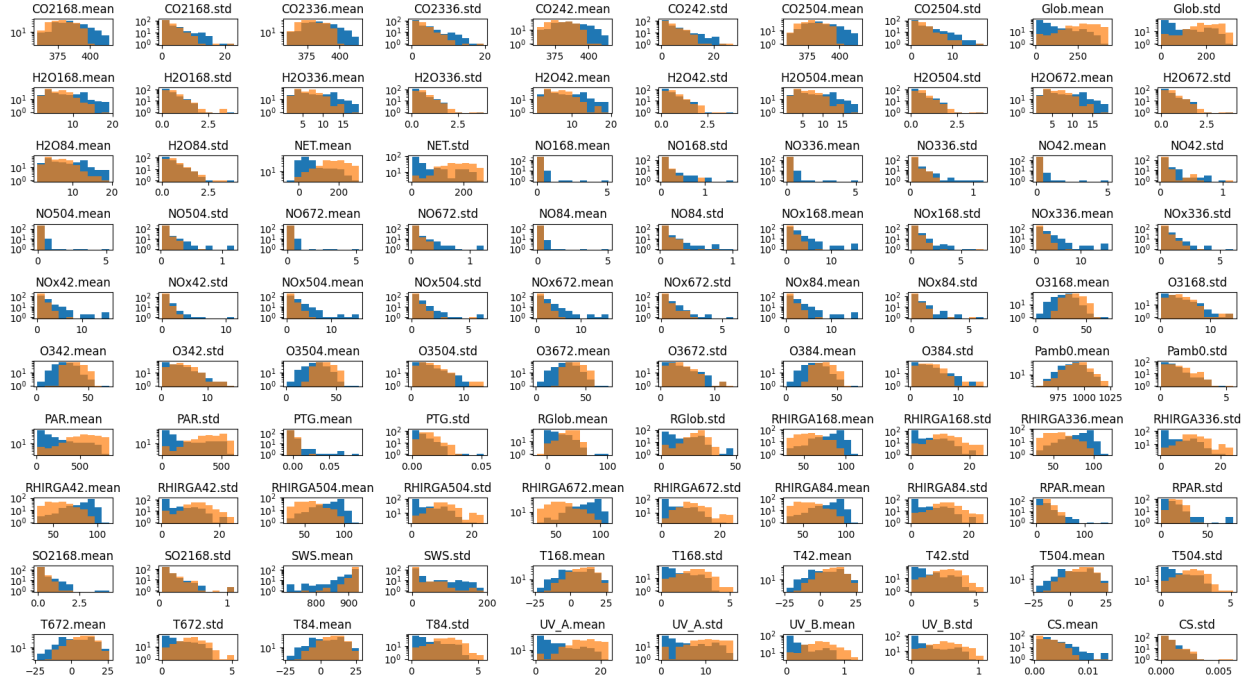


Figure 1. Every training set feature plotted as overlaid **event** and **nonevent** histograms. The blue bars are **nonevent** observations and orange ones **event** observations.

Figure 1 already tells a lot about the data: a large amount of features have Gaussian distributions, but on the other hand, many features are distributed exponentially. Most of the exponential distributions seem to have outlier observations with values way higher than the bulk, all of which belong in the **nonevent** class. Also, some of the paired histograms seem to be right on top of each other, like in the case of the feature **NOx336.std**, suggesting those features maybe aren't going to be as useful in training the classifier and feature selection could improve the model's accuracy.

To help me visualize the distributions of the events only, I plotted a similar chart of them, which you can find in figure 2.

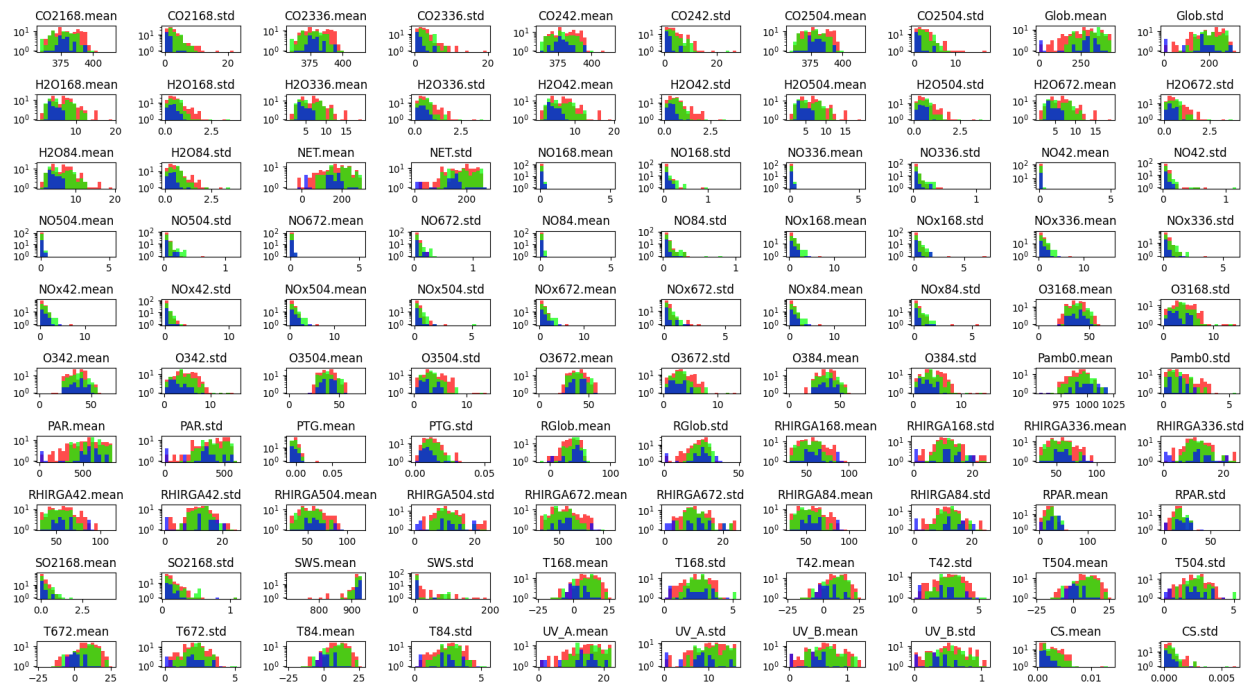


Figure 2. Every training set feature plotted as histograms of observations belonging to each event class. The red bars are **II** events, the green bars are **Ia** events and blue bars **Ib** events.

From figure 2 I can see that the overlaid distributions for each feature are very similar, suggesting that predicting the type of event is much harder than predicting whether an event happens or not. This means I should probably optimize my algorithms to make accurate multi-class prediction instead of focusing on event vs. nonevent.

Planning My Approach

It seems every team that presented their solutions in the final lecture used a hierarchical system where they had one model for predicting **event** or **nonevent** days and another model for predicting the kind of event that happens, if one does. Also from some initial testing, I noticed my multi-class predictions from just one model were not bad. So, purely because seemingly every other team is using a hierarchical system, I want to try to train only one classifier to make predictions for all four classes. I acknowledge that a hierarchical system is almost definitely better, I just want to try something different, since the grade of this project doesn't depend on the best score achieved.

To compare my classifiers I will use an aggregate scoring metric of the following form:

$$\text{score} = \frac{1}{3}[\text{binary accuracy} + \text{multi-class accuracy} + \max(0, \min(1, 2 - \text{perplexity}))],$$

where **binary accuracy** is calculated event vs nonevent, multi-class accuracy also takes into account all the types of events, and perplexity is $e^{\text{mean binary cross-entropy}}$. The same metric was used in the Kaggle competition, which is why I stuck to it.

For a prediction task like this, gradient-boosted decision trees (GBDT) are generally known to be the best, so I will compare the most popular ones. First, however, I will establish a baseline with logistic regression, and try a few other classifier algorithms like support vector machine (SVM) classifier and linear discriminant analysis (LDA). Out of curiosity, I will also try a naive Bayes (NB) classifier and k -nearest neighbors (K-NN), which I don't expect to perform too well, since, unmodified, the features are not independent within-class and there are so many of them.

I will evaluate each algorithm by taking the average score of repeated ten-fold cross-validation runs (using scikit-learn's **StratifiedKFold** with **shuffle=True**) to estimate the generalization performance. Since class **Ib** is quite rare, I have to use stratified folds. I chose ten-fold CV, since it is more stable and the dataset is quite small. I do have to note that not using a nested grid search will result in slightly optimistically biased "test" scores, as I am going to use the validation folds to pick the best hyperparameters. I will also use a **Pipeline** to avoid leakage and more optimistically biased scores. First I will find a decent combination of hyperparameters using a manual grid search for each algorithm, and then see if feature selection, principal components analysis (PCA) or adding polynomial features improves the score. I will also of course standardize the data inside the CV folds before feeding it to the linear models, meaning I will center each column (mean zero) and scale it to unit variance (using scikit-learn's **StandardScaler**).

Next I will try ensembles of the classifiers using the hyperparameters I obtained in the previous step. More specifically I will try each possible ensemble of two and three different models with soft voting to average their predicted probabilities. Using an ensemble will most definitely improve the scoring as the models will help cover each others' weaknesses.

The last step is final tuning of the best ensembles to see if changing the weights improves the score. I left this to the last part to save electricity and time. I will also try clipping the probabilities to make the classifier less confidently incorrect, which the perplexity part of the aggregate score punishes.

Training the Classifiers

In addition to tuning the algorithms' hyperparameters, I tried combinations of the following:

- Standardizing the data (with scikit-learn's **StandardScaler**) inside the **Pipeline**.
- Optimizing the model using an in-built loss function of log-loss, if possible. Hoping maybe this would improve the perplexity score without hurting the accuracies.
- Dropping some data outliers by dropping all rows with feature **N0168.mean** above 0.5, hoping maybe this improves the linear fits.
- Only picking features that most correlate with the target variable (correlation of > 0.2 and > 0.3), mostly to speed up generating polynomial features.
- PCA with $[2, 50]$ components inside the **Pipeline**.
- Algorithm assisted greedy feature selection (using scikit-learn's **SequentialFeatureSelector**, **RFE** and **SelectKBest**) inside the **Pipeline**.
- Generating polynomial features of degree 2 (**interactions_only=True** and **False**).

- Calibrating probabilities for better perplexity scores (with scikit-learn's `CalibratedClassifierCV`), hoping it wouldn't hurt the accuracies too much.

The models were implemented using scikit-learn (and LightGBM/XGBoost/CatBoost libraries where applicable). For linear models the pipeline was `StandardScaler -> classifier`. Tree models were trained without scaling. In the following table I have listed the tuning I achieved the best mean CV score with, for each of the aforementioned classifiers:

Algorithm	CV score	Notes
Logistic Regression*	0.7224	C=0.20
LDA*	0.7264	tol=0.28
Naive Bayes	0.6591	9 principal components
K-NN*	0.7077	k=25
SVM*	0.7161	probability=True, C=3.0
Random forest	0.7126	max_depth=6
scikit-learn's histogram-based GBDT	0.7033	max_depth=1
LightGBM	0.7101	n_estimators=30, colsample_bytree=0.1, reg_lambda=1
CatBoost	0.7036	n_estimators=30, max_depth=4
XGBoost	0.7057	n_estimators=300, learning_rate=0.05, max_depth=1

*The data was standardized inside the Pipeline.

Table 1. Cross-validation scores for different classifiers, alongside the hyperparameters I used to achieve them. My feature selection tuning was quite limited, since the feature selection algorithms are very computationally expensive, especially when combined with heavier classifiers. With this in mind, reducing the amount of features didn't improve the score noticeably. Dropping outliers, using most covariate features, polynomial features or optimizing the classifiers with log-loss as the metric didn't noticeably improve the scores either. Even for naive Bayes, using PCA was better than selecting features. Probability calibration often worsened the scores as the gain in perplexity was offset doubly by a decrease in accuracy.

Looking at table 1, logistic regression and linear discriminant outperform all the GBDT implementations with minimal tuning, though this is most likely due to my inexperience tuning the gradient-boosting trees. Naive Bayes achieved a decent score with PCA, but still the lowest of all the classifiers due to the data violating the assumption of conditional independence within the classes. K -nearest neighbors also reaches a score comparable to the other classifiers by only tuning k to 25. Random forest I tuned extensively but it seems I get the best score by just limiting the depth of the trees to 6. All the gradient-boosting decision tree models achieved similar scores to each other with LightGBM slightly outperforming the others. I spent significantly longer tuning the tree based models than the linear models, but I only iterated through the values of one hyperparameter at a time, so I most likely missed many optimal combinations.

Next I trained some ensemble models and recorded the scores of each possible combination of a total of two and three different models. I used soft voting and equal weights. In table 2 you will find the top 15 scoring ensembles of two and three and their scores:

Ensemble of 2	Score	Ensemble of 3	Score
HGB+LDA	0.7295936	LDA+LGBM+SVM	0.7286261
CB+LDA	0.7291007	CB+K-NN+LDA	0.7286178
LDA+XGB	0.7287313	HGB+LDA+SVM	0.7285603
LDA+SVM	0.7271014	HGB+LDA+LR	0.7285247
LDA+LGBM	0.7267742	LDA+LR+XGB	0.7283863
K-NN+LDA	0.7259248	CB+LDA+LR	0.7278990

Ensemble of 2	Score	Ensemble of 3	Score
LDA+RF	0.7231970	LDA+SVM+XGB	0.7276388
LR+SVM	0.7222781	CB+LDA+SVM	0.7275513
LDA+LR	0.7222087	LDA+LR+SVM	0.7273082
HGB+LR	0.7205107	CB+HGB+LDA	0.7272146
LR+RF	0.7183817	CB+LDA+LGBM	0.7268930
CB+LR	0.7178277	K-NN+LDA+LGBM	0.7268916
LR+XGB	0.7166126	HGB+LDA+LGBM	0.7263794
LR+LGBM	0.7157301	HGB+LDA+RF	0.7263164
CB+HGB	0.7137671	HGB+K-NN+LDA	0.7260407

Table 2. The best scoring ensembles of two (left) and three (right) and their average scores of 20 and 30 random 3-fold CV runs, respectively. The reason I chose three folds was purely to save time. Using only three folds means the scores are a bit lower due to training with less data. All the classifiers were tuned as previously stated and soft voting was used for calculating ensemble probabilities. The initializations stand for the following: LR = logistic regression, LDA = linear discriminant analysis, K-NN = K-nearest neighbors, RF = random forest, SVM = support vector machine, HGB = scikit-learn’s histogram-based GBDT, LGBM = LightGBM, CB = CatBoost, XGB = XGBoost.

When examining table 2, it becomes apparent that LDA seems to be my best linear classifier, as it is present in every single ensemble of three and in all the top seven ensembles of two. Almost every ensemble is a combination of one or more linear models and one tree based model, with boosted trees slightly outperforming random forest. I should also note that the differences between all the top 5 ensembles are significantly smaller than the variance of the individual runs’ scores, which means the ensembles in table 2, especially the ensembles of three, are basically equally good. This is amplified further by the fact that I used three-fold CV, which made the results even more unstable. Therefore, instead of taking the “best” ensembles from table 2, I will compute the average score of 100 runs of each of the ensembles, and only then pick the ensembles I will tune further.

Ensemble of 2	Score	Ensemble of 3	Score
HGB+LDA	0.7266460	LDA+LGBM+SVM	0.7270677
CB+LDA	0.7258189	CB+K-NN+LDA	0.7259244
LDA+XGB	0.7283580	HGB+LDA+SVM	0.7285282
LDA+SVM	0.7274741	HGB+LDA+LR	0.7263929
LDA+LGBM	0.7283852	LDA+LR+XGB	0.7263358
K-NN+LDA	0.7250246	CB+LDA+LR	0.7268182
LDA+RF	0.7258976	LDA+SVM+XGB	0.7267585
LR+SVM	0.7218636	CB+LDA+SVM	0.7287093
LDA+LR	0.7190439	LDA+LR+SVM	0.7273354
HGB+LR	0.7152772	CB+HGB+LDA	0.7269552
LR+RF	0.7163371	CB+LDA+LGBM	0.7268779
CB+LR	0.7186573	K-NN+LDA+LGBM	0.7252695
LR+XGB	0.7164069	HGB+LDA+LGBM	0.7286917
LR+LGBM	0.7197029	HGB+LDA+RF	0.7249847
CB+HGB	0.7130581	HGB+K-NN+LDA	0.7268782

Table 3. The top ensembles’ average scores from a hundred random three-fold stratified cross-validation runs. I chose three folds because I wanted to scores to be comparable to the scores in the previous table, and also to save time.

The scores in table 3 are noticeably different than the scores in table 2, and most importantly, the ensembles are no longer in order of descending scores. I will continue with ensembles LDA+XGB, LDA+LGBM, HGB+LDA+SVM,

CB+LDA+SVM and HGB+LDA+LGBM. Now that I am more confident I have the best ensembles, I will tune the weights and see if that improves the scores.

Ensemble	Ensemble Weights	Aggregate Score	Score std.	Binary accuracy	Multi-class accuracy	Perplexity
LDA+XGB	2/1	0.734555	0.003907	0.882311	0.661600	1.340243
LDA+LGBM	2/1	0.735529	0.004854	0.882977	0.664933	1.341323
HGB+LDA+SVM	2/5/2	0.738266	0.003913	0.884400	0.672177	1.338532
CB+LDA+SVM	1/3/1	0.738750	0.005490	0.886933	0.668933	1.342860
HGB+LDA+LGBM	1/3/1	0.734856	0.005098	0.880133	0.663688	1.339254

Table 4. The seemingly best ensembles from previous tests with their weights tuned and their consequent scores. The scores are averages of 50 random ten-fold stratified cross-validation runs and the standard deviations are the calculated from each CV from the means of the CV runs and not from each CV fold. The weights are applied in the respective order. I also tried clipping the probabilities but none of the scores improved so I chose not to include the optimal clipping strengths.

In table 4 you can find the final scores. These are the best I can do in one week of coding without using a hierarchical system. Calling either the CB+LDA+SVM or the HGB+LDA+SVM ensemble the best of them would probably be false as all the scores are very similar and within one standard deviation of each other. However, CB+LDA+SVM does have the best average aggregate score I got on any single model or ensemble, while HGB+LDA+SVM has the best perplexity score and is more stable.

There were some limitations in addition to only working on this project for a week and choosing not to use a hierarchical model: I only searched for hyperparameters one-at-a-time, so I likely missed good interacting settings. Also, only using one score to determine what is better or not is blunt, since improvements can come from different sources and should be decomposed. Also I didn't discuss which features are the ones driving decisions, which is important with overlapping subclasses. I also didn't record the versions of the software I utilized, which can hurt reproducibility. And lastly I didn't report the standard deviations for all my scores, which could have helped in picking the ensembles to continue with from table 4.

Future Work

The most obvious next step is to implement the hierarchical system that I intentionally avoided, where one model predicts **event/nonevent** and a second model predicts **II**, **Ia** or **Ib** conditional on an event. Since the distributions of the event subclasses overlap heavily, a hierarchical approach is very likely to improve the multi-class part of the aggregate score, and it may also help reduce the confidently incorrect predictions that hurt the perplexity metric.

Another improvement would be more systematic hyperparameter tuning. Here I mostly tuned one hyperparameter at a time and under a somewhat tight time budget, which is especially limiting for the gradient-boosting tree models. Random search or Bayesian optimization over multiple interacting hyperparameters would be more appropriate.

Since **Ib** is rare, I would also address the class imbalance. This includes trying class-weighted losses, different sampling approaches, and reporting confusion matrices in addition to the aggregate score. This would also make it clearer whether the models are improving the hard part of the task (event type prediction) or mostly just improving event/nonevent detection.

Also, something I had not considered before the final lecture was interpreting the data as showing the chemistry of the atmosphere. This would have allowed me to build stronger features, which in turn would most have most likely improved the score further.

Finally, because the score includes a probability-quality component, I would revisit calibration: I would compare different calibration methods and report their calibration curves. This would clarify whether probability clipping or calibration can reliably improve the perplexity component without sacrificing accuracy.

Conclusion

The objective of this project was to predict new particle formation outcomes independently of time, and I chose to train a single classifier to predict all four classes (**nonevent**, **II**, **Ia**, **Ib**) instead of using a hierarchical system. Exploratory plots showed that many features are roughly Gaussian while others are exponential with extreme outliers largely in the **nonevent** class. When restricting attention to events only, the subclass feature distributions overlap heavily, which suggests that predicting the event type is substantially harder than predicting whether an event occurs.

Using the Kaggle-style aggregate score combining binary accuracy, multi-class accuracy and a perplexity-based term, I compared a set of linear models, instance-based methods, and several gradient-boosted decision tree implementations under repeated ten-fold stratified cross-validation. With limited tuning, logistic regression and LDA performed strongly, while the boosted tree models achieved similar scores to each other but did not outperform the best linear baselines. Soft-voting ensembles improved results however, and after selecting a small set of top ensembles and tuning voting weights, the best average score I achieved was about 0.739 with the **CB+LDA+SVM** ensemble. However, the differences among the top ensembles were very small relative to the variance across random CV runs, so it is more accurate to treat the best few ensembles as essentially tied given the current evaluation budget.

Overall, this report shows that well-tuned linear models and small ensembles can provide strong baselines for such tasks. Also, I should note that I was not surprised to get a significantly lower score than the teams presenting in the final lecture, since they all used a hierarchical system.

References

- IML25: Classifying New Particle Formation: <https://www.kaggle.com/competitions/iml25-term-project>
- Breiman, L. (2001). Random Forests. *Machine Learning*.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD*.
- Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*.
- Prokhorenkova, L., et al. (2018). CatBoost: unbiased boosting with categorical features. *NeurIPS*.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*.